



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ
НАУКА У НОВОМ САДУ



Проф. др Игор Дејановић

Шаблон и упутство за писање завршних радова

Нови Сад, 2026

	УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА 21000 НОВИ САД, Трг Доситеја Обрадовића 6
	КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР:									
Идентификациони број, ИБР:									
Тип документације, ТД:	Монографска документација								
Тип записа, ТЗ:	Текстуални штампани материјал								
Врста рада, ВР:	Дипломски - мастер рад								
Аутор, АУ:	Уписати име и презиме								
Ментор, МН:	Др Игор Дејановић, редовни професор								
Наслов рада, НР:	Шаблон и упутство за писање завршних радова								
Језик публикације, ЈП:	српски/хирилица								
Језик извода, ЈИ:	српски/енглески								
Земља публиковања, ЗП:	Република Србија								
Уже географско подручје, УГП:	Војводина								
Година, ГО:	2026								
Издавач, ИЗ:	Ауторски репринт								
Место и адреса, МА:	Нови Сад, трг Доситеја Обрадовића 6								
Физички опис рада, ФО: (поглавља/страна/ цитата/табела/слика/графика/прилога)	3/19/5/0/0/0/2								
Научна област, НО:	Електротехничко и рачунарско инжењерство								
Научна дисциплина, НД:	Примењене рачунарске науке и информатика								
Предметна одредница/Кључне речи, ПО:	Шаблон, завршни рад, упутство								
УДК									
Чува се, ЧУ:	У библиотеци Факултета техничких наука, Нови Сад								
Важна напомена, ВН:									
Извод, ИЗ:	Овај документ представља упутство за писање завршних радова на Факултету техничких наука Универзитета у Новом Саду. У исто време је и шаблон за Turpst.								
Датум прихватања теме, ДП:									
Датум одбране, ДО:	01.01.2025								
Чланови комисије, КО:	<table border="1"> <tr> <td>Председник:</td> <td>Др Петар Петровић, ванредни професор</td> </tr> <tr> <td>Члан:</td> <td>Др Марко Марковић, доцент</td> </tr> <tr> <td>Члан:</td> <td></td> </tr> <tr> <td>Члан, ментор:</td> <td>Др Игор Дејановић, редовни професор</td> </tr> </table>	Председник:	Др Петар Петровић, ванредни професор	Члан:	Др Марко Марковић, доцент	Члан:		Члан, ментор:	Др Игор Дејановић, редовни професор
Председник:	Др Петар Петровић, ванредни професор								
Члан:	Др Марко Марковић, доцент								
Члан:									
Члан, ментор:	Др Игор Дејановић, редовни професор								
	Потпис ментора								

	UNIVERSITY OF NOVI SAD • FACULTY OF TECHNICAL SCIENCES 21000 NOVI SAD, Trg Dositeja Obradovića 6	
	KEY WORDS DOCUMENTATION	
Accession number, ANO :		
Identification number, INO :		
Document type, DT :	Monographic publication	
Type of record, TR :	Textual printed material	
Contents code, CC :		
Author, AU :	Upisati ime i prezime na latinici	
Mentor, MN :	Igor Dejanović, Phd., full professor	
Title, TI :	Template and tutorial for thesis preparation	
Language of text, LT :	Serbian	
Language of abstract, LA :	Serbian/English	
Country of publication, CP :	Republic of Serbia	
Locality of publication, LP :	Vojvodina	
Publication year, PY :	2026	
Publisher, PB :	Author's reprint	
Publication place, PP :	Novi Sad, Dositeja Obradovica sq. 6	
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	3/19/5/0/0/0/2	
Scientific field, SF :	Electrical and Computer Engineering	
Scientific discipline, SD :	Applied computer science and informatics	
Subject/Key words, S/KW :	Template, thesis, tutorial	
UC		
Holding data, HD :	The Library of Faculty of Technical Sciences, Novi Sad	
Note, N :		
Abstract, AB :	This document provides guidelines for writing final theses at the Faculty of Technical Sciences, University of Novi Sad. At the same time, it serves as a Typst template.	
Accepted by the Scientific Board on, ASB :		
Defended on, DE :	01.01.2025	
Defended Board, DB :	President:	Petar Petrović, Phd., assoc. professor
	Member:	Marko Marković, Phd., asist. professor
	Member:	
	Member, Mentor:	Igor Dejanović, Phd., full professor
		Menthor's sign



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат

Садржај

1	Увод	1
1.1	Мотивација	2
1.2	Предмет и циљ рада	2
1.3	Основне функционалности алата RustyJudge	3
1.4	Организација рада	4
2	Стање у области	7
3	Закључак	9
	Списак коришћених скраћеница	11
	Списак коришћених појмова	13
	Биографија	15
	Литература	17
	Грешке у литературним наводима	19

Савремени софтверски системи се у великој мери ослањају на језике и алате који омогућавају брз развој, једноставну интеграцију и извршавање на различитим платформама. Један од најзначајнијих језика у том контексту је JavaScript, који се користи у развоју веб апликација, серверских система, алата командне линије, десктоп апликација и проширења за развојна окружења. Због широке примене, JavaScript код често настаје у великим и дуговечним пројектима, у којима га мења више програмера и у којима грешке у коду могу да имају значајан утицај на поузданост, одрживост и разумљивост система.

Флексибилност JavaScript језика истовремено представља и предност и изазов. Језик дозвољава различите стилове програмирања, динамичку природу вредности, слободнију употребу променљивих и велики број синтаксичких конструкција. Таква флексибилност убрзава развој, али може да отежа уочавање грешака пре извршавања програма. Неке неправилности су синтаксичке и лако се откривају већ током парсирања кода. Међутим, велики број проблема није видљив само из појединачне синтаксичке конструкције. За њихово откривање потребно је разумети односе између наредби, ток извршавања програма, области важења идентификатора и начин на који се вредности дефинишу и користе.

Један од приступа за рано откривање таквих проблема је статичка анализа програмског кода. Статичка анализа обухвата поступке у којима се програм испитује без његовог покретања. У пракси се ова идеја често примењује кроз алате познате као linter-и. Linter-и анализирају изворни код и пријављују потенцијалне грешке, сумњиве обрасце, некоришћене делове програма, непожељан стил писања или конструкције које могу да доведу до тежег одржавања. За JavaScript постоје зрели алати као што су ESLint и RSLint, али је за разумевање начина на који такви алати раде корисно развити сопствену имплементацију која експлицитно приказује све важне фазе анализе.

У овом раду приказан је развој алата RustyJudge, linter-a за JavaScript написаног у програмском језику Rust. Алат је осмишљен као статички анализатор који, поред основне синтаксичке обраде, гради унутрашње представе програма потребне за сложенија правила провере. Посебна пажња посвећена је изградњи контролно-проточног графа (CFG, енг. *Control-Flow Graph*), анализи достижности делова програма и анализи живости података (енг. *liveness analysis*). На основу тих информација мо-

гуће је имплементирати правила која не зависе само од облика једне синтаксичке конструкције, већ и од начина на који се програм може извршавати.

1.1 Мотивација

Основна мотивација за развој алата RustyJudge произилази из потребе да се поступак статичке анализе JavaScript кода прикаже као целовит процес. У једноставном случају, linter може да обиђе апстрактно синтаксичко стабло (AST, енг. *Abstract Syntax Tree*) и да провери да ли се у коду појављује одређена синтаксичка форма. Такав приступ је довољан за правила која, на пример, забрањују одређени оператор, проверавају именовање идентификатора или препознају једноставан образац у изразу.

Међутим, бројна правила захтевају више информација. Ако је потребно утврдити да ли је део кода недостижан, није довољно посматрати само једну наредбу. Потребно је знати шта се дешава пре ње, да ли претходна наредба прекида ток извршавања и које гране програма могу да доведу до посматраног места. Слично томе, за откривање променљивих чија вредност више није потребна није довољно знати само где је променљива декларисана. Потребно је анализирати где се вредност дефинише, где се користи и да ли постоји путања извршавања на којој је та вредност и даље релевантна.

Због тога је у RustyJudge-у нагласак стављен на представе које се често користе у компајлерима и статичким анализаторима. AST служи као почетна структура добијена након парсирања изворног кода. Над њим се затим граде додатне информације, као што су симболи, области важења, CFG и скупови потребни за анализу живости. На тај начин се linter не посматра само као скуп независних провера, већ као алат који пролази кроз више фаза анализе и постепено обогаћује разумевање програма.

Програмски језик Rust изабран је због следећих акрактеристика. Погодан за развој системских и развојних алата зато што омогућава високе перформансе, експлицитно управљање подацима и строжију контролу над исправношћу програма у време компилације. За алат који обрађује изворни код, гради графове и покреће више правила анализе, ове особине су значајне јер утичу на брзину извршавања, организацију података и поузданост имплементације.

1.2 Предмет и циљ рада

Предмет рада је пројектовање и имплементација алата RustyJudge за статичку анализу JavaScript кода. Алат чита изворне датотеке, парсира их, гради унутрашње структуре потребне за анализу, покреће скуп правила и кориснику приказује дијагностике. Дијагностика представља поруку о уоченом проблему, заједно са позицијом у изворном коду и описом правила које је прекршено.

Циљ рада је да се прикаже како се један linter може изградити као компајлерски оријентисан алат. То подразумева да се имплементација не заснива само на директном обиласку синтаксичког стабла, већ и на изградњи богатијих модела програма. Посебни циљеви рада су:

- приказ поступка парсирања JavaScript кода и добијања AST структуре;
- изградња семантичког контекста са информацијама о симболима и областима важења;
- изградња CFG представе погодне за анализу тока извршавања;
- имплементација анализе достижности и анализе живости података;
- дефинисање инфраструктуре за извршавање правила;
- имплементација конкретних правила linter-a;
- приказ дијагностика кроз CLI, LSP и VS Code проширење;
- поређење перформанси са алатима ESLint и RSLint.

Овако постављен циљ омогућава да се RustyJudge посматра из два угла. Са једне стране, он је практичан алат који може да анализира JavaScript датотеке и прикаже кориснику проблеме у коду. Са друге стране, он је пример примене концепата из области компајлера и статичке анализе на конкретан програмски језик и конкретан развојни алат.

1.3 Основне функционалности алата RustyJudge

RustyJudge је организован као Rust пројекат који садржи библиотечки део, извршне улазе за CLI и LSP, скуп правила, модуле за CFG и анализу података, benchmark окружење и VS Code проширење. Основни ток рада почиње читањем JavaScript изворног кода и његовим парсирањем. Након тога се над добијеном структуром покрећу анализе и правила која производе дијагностике.

Прва група функционалности односи се на синтаксичку обраду и обилазак AST-а. Овај део омогућава да се препознају језичке конструкције као што су декларације, изрази, функције, условне наредбе и петље. На основу такве структуре могуће је имплементирати правила која зависе од непосредног облика кода.

Друга група функционалности односи се на семантичку анализу. У овом делу се прикупљају информације о симболима, декларацијама и областима важења. Те информације су важне за правила која морају да разликују појаву имена у коду од стварне променљиве или функције на коју се то име односи. Без таквог слоја анализе, linter би могао да препозна само текстуално исто име, али не и његово значење у одређеном делу програма.

Трећа група функционалности односи се на изградњу CFG-а. CFG представља програм као скуп чворова и грана које описују могући ток извршавања. Ова структура је посебно значајна за откривање недостижног кода, анализу гранања и припрему

података за касније анализе. Уместо да се наредбе посматрају само редом којим су написане, CFG омогућава да се експлицитно прикажу путање које програм може да прати током извршавања.

Четврта група функционалности односи се на анализу живости података. Ова анализа испитује да ли је вредност неке променљиве потребна у наставку извршавања програма. На основу скупова дефиниција и употреба могуће је израчунати које су променљиве живе на улазу и излазу из појединих делова CFG-а. Такве информације могу да се користе за правила која откривају непотребна додељивања, некоришћене вредности или друге проблеме у вези са током података.

Поред анализе у командној линији, RustyJudge садржи и интеграцију са развојним окружењем. LSP сервер омогућава да се резултати анализе прикажу у едитору, док VS Code проширење служи као клијентска страна која повезује корисника, отворену датотеку и RustyJudge анализатор. На тај начин се дијагностике могу приказати директно на месту где програмер пише код, што је природан начин употребе linter-а у свакодневном развоју.

Значајан део рада представља и benchmark анализа. Пошто постоје већ познати алати за анализу JavaScript кода, RustyJudge је поређен са ESLint-ом и RSLint-ом. Ово поређење не служи само да покаже време извршавања, већ и да се објасни контекст мерења: које су датотеке анализиране, која правила су покренута, како су алати конфигурисани и која су ограничења таквог поређења.

1.4 Организација рада

Рад је организован тако да се од општих појмова постепено прелази ка конкретној имплементацији алата RustyJudge. У другом поглављу приказани су појам статичке анализе, улога linter-а у развоју софтвера и постојећи алати за анализу JavaScript кода, са посебним освртом на ESLint и RSLint.

У трећем поглављу изложене су теоријске основе потребне за разумевање имплементације. Описани су AST, области важења, CFG, основни блокови, достижност, ток података и анализа живости. Ово поглавље поставља основу за касније разумевање начина на који RustyJudge представља и анализира програм.

Четврто поглавље описује архитектуру RustyJudge-а. Приказан је ток података од улазне JavaScript датотеке до дијагностике, као и улога главних модула у систему. Пето поглавље посвећено је имплементацији CFG-а и начину на који се поједине JavaScript конструкције преводе у граф тока извршавања.

У шестом поглављу описана је анализа живости података и њена примена у оквиру алата. Седмо поглавље приказује инфраструктуру за правила linter-а и конкретна

правила која су имплементирана. Осмо поглавље описује употребу алата кроз CLI, LSP и VS Code проширење.

Девето поглавље посвећено је евалуацији и поређењу перформанси са алатима ESLint и RSLint. У њему су приказани услови мерења, резултати и ограничења поређења. На крају, у закључку су сумирани резултати рада, наведена ограничења тренутне имплементације и предложени правци даљег развоја.

Глава 2

Стање у области

Даља поглавља садрже опис стања у области, коришћене технологије, опис дизајна и имплементације итд. Најбоље је да свако поглавље пишете у посебном `.typ` фајлу. Не заборавите да га укључите у главом фајлу `završni-rad.typ` (претражити све `TODO` коментаре).

Глава 3

Закључак

У закључку дајте кратак преглед онога шта урађено, са освртом на проблеме који су решени, предности и мане решења и правце даљег развоја.

Списак коришћених скраћеница

Скраћеница	Опис
API	Application Programming Interface (апликациони програмски интерфејс)
AWS	Amazon Web Services (Амазон веб сервиси)
CI/CD	Continuous Integration / Continuous Delivery (континуирана интеграција / континуирана испорука)
CORS	Cross-Origin Resource Sharing (размена ресурса између извора и дестинације различитог порекла)
CSS	Cascading Style Sheets (језик за описивање стилова)
DOM	Document Object Model (објектни модел документа)
DTO	Data Transfer Object (објекат за пренос података)
HTTP	HyperText Transfer Protocol (протокол за пренос хипертекста)
JSON	JavaScript Object Notation (формат за размену података)
JWT	JSON Web Token (сигурносни токен заснован на JSON формату)
RLS	Row-Level Security (сигурност на нивоу реда)
REST	Representational State Transfer (скуп правила за комуникацију између клијента и сервера)
RPC	Remote Procedure Call (позив удаљене процедуре)
SQL	Structured Query Language (структурирани упитни језик)
TLS	Transport Layer Security (безбедност транспортног слоја)
UML	Unified Modeling Language (језик за моделовање дијаграма)
URL	Uniform Resource Locator (јединствени идентификатор и локатор ресурса)
UI	User Interface (кориснички интерфејс)
UUID	Universally Unique Identifier (универзално јединствени идентификатор)
WAL	Write-Ahead Logging (записивање операција унапред)

Списак коришћених појмова

Појам	Објашњење
Асинхрони рад	Рад који се изводи независно од главног тока извршавања, омогућавајући наставак других операција без чекања на његов завршетак.
Bucket (S3)	Логичка јединица за складиштење у AWS S3 сервису, која организује фајлове у облаку.
Read-Only	Режим рада у коме су подаци само за читање, без могућности измене.
Cross-platform	Способност софтвера да се извршава на више различитих оперативних система из истог кода.
Connection pool	Механизам за управљање и поновну употребу веза са базом података како би се побољшале перформансе апликације.

Биографија

Овде написати своју кратку биографију.

Литература

Грешке у литературним наводима

Овде се налази списак грешака у литературним наводима које морате исправити у `literatura.bib` фајлу.

1. Референци `pyflies` недостаје поље `urldate`